Project: Authentication & API Gateway

Due: 03/21/2026 11:59 PM
Extended Deadline: 03/22/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Ingress + Gateway
- Task Overview
- Task

KrakenD
- Task Overview
- Task

Django Authentication
- Task Overview
- Task

Securing LLM API
- Task Overview
- Task

Wrap Up
- Conclusion

KrakenD (Task)

Project Architecture

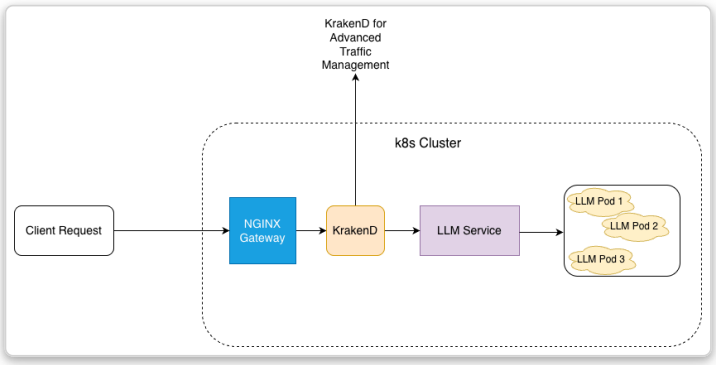


Figure: Task 2 Architecture Diagram

Take some time to review the diagram and familiarize yourself with each component. Understanding how these pieces work together will make it much easier to follow the remaining steps of the project.

With your NGINX Gateway, you can now access your microservice in much the same way Azure Functions do. However, while Gateways are powerful, they do not provide built-in mechanisms for authentication or access management. Some Gateway API controllers support mutual TLS, allowing trusted clients to present SSL certificates for verification, but this support is limited and still requires manual certificate management.

As you can imagine, this quickly becomes cumbersome. If we want to expose our services to external consumers—other teams or even the public—this approach does not scale well. Moreover, while Gateway APIs help control the networking layer of an application (how traffic is routed and handled), they do not manage the developer-facing aspects of an API—such as request structure, validation, or versioning.

To scale effectively, we need more than just a load balancer; we need an API management solution. In cloud-native terms, this is known as an API Gateway.

API Gateways provide significantly more control over how APIs are exposed and consumed. They typically offer:

- **API Portals:** Developer-facing interfaces to explore, monitor, and understand available APIs.
- **Mock API Responses:** The ability to simulate endpoints, helping with API design and testing before implementation.
- **Basic Authentication & Access Control:** Fine-grained control over who can access specific APIs.
- **API Monitoring:** Visibility into service health, uptime, and request behavior.
- **API Versioning:** Support for evolving APIs while maintaining backward compatibility.

For this task, we will use **KrakenD**, an open-source API Gateway project under the Linux Foundation. Azure also offers a managed alternative called **Azure API Management**, but it is less suited for Kubernetes-native workflows. In our setup, KrakenD will act as an intermediary layer between the NGINX Gateway Fabric and the backend pods.

Before proceeding, it's important to distinguish between **KrakenD Community Edition** and **KrakenD Enterprise Edition**. We will use the Community Edition for this course. The Enterprise Edition includes additional features and enterprise-grade capabilities that may be useful in production environments, depending on your needs.

Subtask 1: Configure KrakenD Using the Web Portal

KrakenD is a stateless API gateway. At this point, you might wonder: if KrakenD is stateless, how does it store all the configuration for our API endpoints? The answer lies in its configuration file. Throughout this project, we will refer to this file as `krakend.json`.

All KrakenD configuration is defined declaratively in this JSON file. Using KrakenD is essentially a two-step process:

1. **Create the configuration file.**
   KrakenD provides a web-based interface called the **KrakenD Designer**, which makes it easy to define routes, policies, and settings. You can then export this configuration as a `krakend.json` file, avoiding the need to manually write JSON.
2. **Provide the configuration to KrakenD running in Kubernetes.**
   Once the file is supplied to the KrakenD deployment, the gateway uses it to determine its behavior.

That's it. Any time you want to change how the gateway behaves, you simply update the JSON file and restart the pod.

Before using the KrakenD Designer, we should install the **KrakenD CLI**. This tool is extremely useful for validating and auditing your configuration files.

Follow the official documentation to install the KrakenD CLI on your student VM:

KrakenD CLI Installation:
https://www.krakend.io/docs/overview/installing/

After installation, verify it by running:

```
krakend version
```

Next, create a simple `krakend.json` file with the following contents. This file is only for demonstration purposes and is not part of the graded task.

```
{
    "$schema": "https://www.krakend.io/sch
    "version": 3,
    "endpoints": [],
    "extra_config": {}
}
```

This is the most minimal KrakenD configuration possible—it does absolutely nothing. However, it is sufficient to demonstrate the difference between `krakend validate` and `krakend audit`.

Validating the Configuration

The `krakend validate` command checks whether your configuration file is syntactically correct.

Run:

```
krakend validate --config krakend.json
```

You should see:

```
Parsing configuration file: krakend.json
Syntax OK!
```

This confirms that the file is valid, even though it contains no real functionality.

Auditing the Configuration

The `krakend audit` command performs a rule-based analysis of your configuration and highlights potential issues or best-practice violations.

Run:

```
krakend audit --config krakend.json
```

You will see output similar to the following:

```
1.2.1   [HIGH]    Prioritize using JWT for
2.1.2   [HIGH]    Enable TLS or use a term
2.1.7   [HIGH]    Enable HTTP security hea
2.2.1   [MEDIUM]  Hide the version banner
2.2.2   [HIGH]    Enable CORS.
3.1.1   [LOW]     Enable a bot detector.
3.1.2   [HIGH]    Implement a rate-limitin
3.1.3   [HIGH]    Protect your backends wi
4.1.1   [MEDIUM]  Implement telemetry for
4.2.1   [MEDIUM]  Enable distributed traci
4.3.1   [MEDIUM]  Use improved logging for
5.1.7   [MEDIUM]  Avoid using sequential p
5.2.2   [LOW]     Use the backend-for-fron
5.2.3   [LOW]     Avoid overusing no-op en
```

As you can see, the audit tool is intentionally conservative. It does not understand your deployment context and therefore flags many potential issues that may not actually apply to your use case.

As a cloud-native developer, it is your responsibility to decide which recommendations are relevant and which are not.

For this task, we will focus on **one specific requirement**:

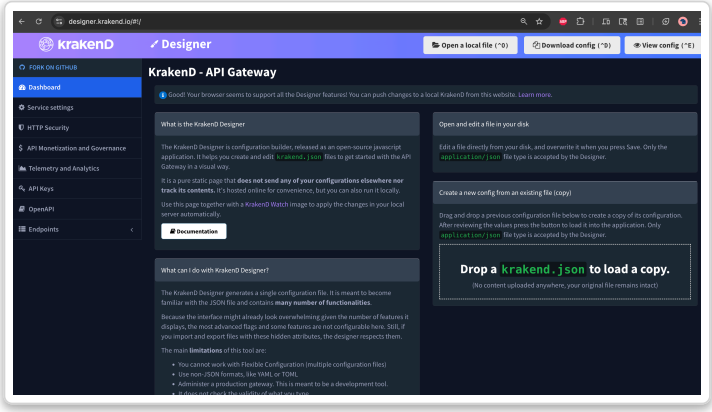
3.1.2 — Implement a rate-limiting strategy

Our API is intentionally fragile, and we want to ensure that only a single user can access it at a time. The API Gateway will act as a protective layer, preventing excessive or unintended access from external clients.

Using the Designer

Go to https://designer.krakend.io/#!/. We will use this web interface to create our configuration file and export it as JSON.

On the left-hand side of the KrakenD Designer UI, you will see a sidebar with **Dashboard** at the top.



Start by clicking **Service Settings**.

1. Enable the **debug endpoints** so that we can troubleshoot KrakenD more easily if something goes wrong during deployment.
2. On the right-hand side, locate the **Hosts** section. These are the backend services that KrakenD can forward requests to.
   Add the following entry:

   ```
   http://YOUR_LLM_SERVICE_NAME
   ```

   Replace `YOUR_LLM_SERVICE_NAME` with the name of your LLM service, which tells KrakenD where to route requests. Do not confuse this with the domain name.

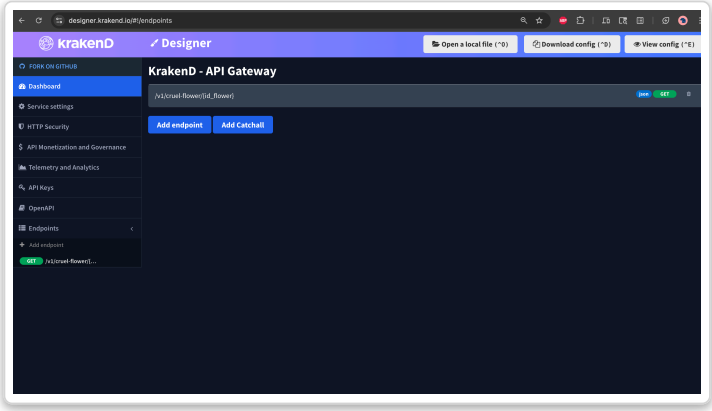
To find your LLM service name, run `kubectl get svc`. You could use the service's Cluster IP instead, but this is

a bad idea — Cluster IPs can change when a cluster is destroyed and recreated, so it's more reliable to let KrakenD do service discovery using the service name.

A common point of confusion is why KrakenD doesn't need a load balancer IP or domain name to reach the LLM service. The reason is simple: both KrakenD and the LLM service are running in the same cluster, so they can communicate directly with each other without going through an external load balancer.

Next, click **Endpoints** in the sidebar and then click **Add Endpoint**. You will be taken to a screen similar to the one shown in the example.

Click the blue endpoint path (for example, `/v1/darkling-garage/{}`) to open the route editor. To connect our service to KrakenD, make the following changes:



Under **KrakenD Endpoint**, change the endpoint path to:

```
/v1/llm
```

The `v1` prefix represents the API version and allows future versioning if needed.
Under **Parameter Forwarding**, add `message` to the **Recognized query string parameters**.
By default, KrakenD does not forward request parameters unless explicitly specified.

Next, open the **Throttling** tab (next to *Parameter Forwarding* and *Auth*).

Enable **Endpoint rate limit**.
Set the **capacity** to `1`.

This ensures that only one request can be processed at a time, preventing overload on the single backend pod.

In the same tab, locate **Timeout and Cache TTL**:

Set **timeout** to `300s`
Set **cache_ttl** to `100s`

Since the LLM service is slow, these values prevent KrakenD from timing out prematurely.

Now open the **Response Manipulation** tab.

Enable **Static Response**
Set the strategy to **Errored**

This ensures that KrakenD returns a meaningful error response if the backend fails. Use the following static response:

```
{
    "choices": [
        {
            "finish_reason": "length",
            "index": 0,
            "logprobs": null,
            "text": "Beep boop I'm a fake
        }
    ],
    "created": 1708201942,
    "id": "cmpl-a",
    "model": "fake_model",
    "object": "text_completion",
    "usage": {
        "completion_tokens": 999,
        "prompt_tokens": 10,
        "total_tokens": 10
    }
}
```

Next, scroll down to the **Backends (Upstream) for /v1/llm** section.

Under **Connection**, select the host you added earlier:

```
http://YOUR_LLM_SERVICE_NAME/
```

Change the **URL** field to:

```
/api
```

At this point, your KrakenD configuration is complete.

Click **Download configuration** in the top-right corner and upload the generated file to your student VM.

Finally, move the file into the `k8s-components` directory and validate it using the KrakenD CLI, just as you did in the previous section.

Subtask 2: Deploy KrakenD with a LoadBalancer

In Kubernetes, a common way to provide configuration files to pods is through **ConfigMaps**, and KrakenD supports this approach. We can create a ConfigMap directly from the `krakend.json` file using the following command:

```
kubectl create configmap krakend-cfg --fro
```

Next, save the provided **Deployment** and **Service** YAML files in the `k8s-components` folder and apply them.

```
# api-gateway.yaml
# KrakenD deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: krakend-deploy
  labels:
    application: apigateway
spec:
  replicas: 1
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  minReadySeconds: 5
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      application: apigateway
  template:
    metadata:
      labels:
        application: apigateway
    spec:
      automountServiceAccountToken: false
```

```
      volumes:
        - name: krakend-cfg
          configMap:
            name: krakend-cfg
      containers:
        - name: application
          resources:
            requests:
              memory: '64Mi'
              cpu: '100m'
            limits:
              memory: '1024Mi'
              cpu: '1000m'
          image: krakend:2.12.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          volumeMounts:
          - name: krakend-cfg
            mountPath: /etc/krakend

---
# LoadBalancer Service
apiVersion: v1
kind: Service
metadata:
  name: apigateway
  labels:
    application: apigateway
spec:
  type: LoadBalancer
  selector:
    application: apigateway
  ports:
    - port: 8080
      targetPort: 8080
```

A few notes about the YAML:

We intentionally use a **LoadBalancer** here so you don't need to integrate with your Gateway yet. In practice, rolling out changes incrementally like this is usually the safest and fastest way to debug.
The Deployment includes a **volumeMount** section, which mounts files into the container at runtime. This is an alternative to building a custom Docker image just to include configuration files.

The **volumes** section mounts the `krakend-cfg` ConfigMap into the pod, effectively copying the `krakend.json` file into the running container.

> **Information**
>
> The KrakenD service must be named `apigateway` and should be accessible from within the cluster on port `8080`.

After applying the Deployment and service YAMLs, check the logs for the KrakenD pod:

1. Find the pod name:

   ```
   kubectl get pods
   ```

2. View logs:

   ```
   kubectl logs POD_NAME
   ```

   Optionally, add `--follow` to stream logs in real time:

   ```
   kubectl logs --follow POD_NAME
   ```

Next, retrieve the external LoadBalancer IP:

```
kubectl get svc
```

Once you have the external IP, test the endpoint in your browser or via `curl` (KrakenD listens on port `8080`):

```
curl "http://LOAD_BALANCER_IP:8080/v1/llm?
```

Debugging Tips for This Section

If you modify `krakend.json`, recreate the ConfigMap:

```
kubectl delete configmap krakend-cfg &&
```

To restart KrakenD cleanly without deleting resources, use a rollout restart:

```
kubectl rollout restart deployment/krake
```

Always check pod logs while debugging—KrakenD typically logs meaningful errors that point to misconfigurations.

If you need more verbose logs, update your `krakend.json` using the Designer:

- Go to **Telemetry and Analytics**
- Enable **Logging**
- Set the log level from **ERROR** to **DEBUG**

This will provide significantly more output to help you understand what KrakenD is doing at runtime.

Subtask 3: Connecting NGINX to KrakenD

Now we need to connect our NGINX Gateway Fabric to KrakenD.

Information

Before you follow the steps below, think through what we have deployed so far and what needs to change.

Our goals are:

1. Update the Gateway routing so it forwards traffic to KrakenD.
2. Update KrakenD so it no longer needs a public LoadBalancer.

Two Kubernetes resources need changes. In what order should you apply them so debugging is easiest? Does the order matter?

Connecting the Gateway

Start by updating the **HTTPRoute** you created in Task 1.

You will need to modify:

- The **path match** so traffic for `/v1/llm` is routed correctly. Refer to the official documentation on how to do this.
- The **backendRefs** so the route points to the KrakenD Service instead of the original LLM service. Don't forget to update the port to use the port the KrakendD sercive is running on.

Apply the updated YAML.

Once this is done, you should be able to access `/v1/llm` through your domain name (via the NGINX Gateway), for example:

```
curl "https://YOUR_DOMAIN/v1/llm?message=w
```

Note that you should curl the endpoint using https and not http.

Hiding the API Gateway

After you confirm the end-to-end flow works through your domain, update the KrakenD Service.

In the KrakenD Service YAML (from Subtask 2), change:

`type: LoadBalancer` → `type: ClusterIP`

Apply the updated Service YAML.

Then re-test the full flow again through your domain name to confirm everything still works.

Submit Your Work

Cluster State

At the end of this task, your cluster state should look something like this:

```
azureuser@qa-vm:~/cloud-native-handout-mik
NAME                                  READY
krakend-deploy-58797545c5-mgfm6       1/1
llmapp-deployment-794767976f-d4jgs    1/1

azureuser@qa-vm:~/cloud-native-handout-mik
NAME              TYPE        CLUSTER-IP
apigateway        ClusterIP   10.0.13.103
kubernetes        ClusterIP   10.0.0.1
llmapp-service    ClusterIP   10.0.101.215

azureuser@qa-vm:~/cloud-native-handout-mik
NAME             CLASS   ADDRESS          P
nginx-gateway    nginx   20.127.176.146   T

azureuser@qa-vm:~/cloud-native-handout-mik
NAME        HOSTNAMES
llmroute    ["s26cloudnative.myvnc.com"]
```

Tree Structure for Submission

At the end of this task, your k8s-components directory should look like this:

```
k8s-components/
├── api-gateway.yaml
├── clusterissuer.yaml
├── gateway.yaml
├── httproute.yaml
└── krakend.json

1 directory, 5 files
```

As long as your configuration matches what is expected and your API responds correctly at `/v1/llm`, you should be good.

Run:

```
./submitter -t task2
```

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform